1 L2–3 Supervised Learning

1.1 Classification and Loss

ERM: population risk $\theta^\star = \arg\min_\theta \mathbb{E}_{x\sim P}\,\mathrm{err}(f(x;\theta),y(x))$; empirical risk $\hat\theta = \arg\min_\theta \frac{1}{N}\sum_i \mathrm{err}(f(x_i;\theta),y_i)$. **Binary logistic**: $p(y=1|x)=\sigma(w^\top x+b)$,

$$\ell = -y\log p - (1-y)\log(1-p), \quad \frac{\partial \ell}{\partial z} = p - y.$$

Multi-class: logits $z\in\mathbb{R}^K$, $p_k = e^{z_k}/\sum_j e^{z_j}$,

$$\ell(z,y) = -\log p_y = -z_y + \log\sum_j e^{z_j}, \quad \frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}[k=y].$$

One-hot CE: $-\sum_k y_k \log p_k$.

1.2 MLP and Backprop

Layer: $h^{(0)}=x$, $z^{(\ell)} = W^{(\ell)}h^{(\ell-1)}+b^{(\ell)}$, $h^{(\ell)}=\phi(z^{(\ell)})$. Batch code convention often uses $X\in\mathbb{R}^{B\times n}$, $Z = XW+b$, $W\in\mathbb{R}^{n\times m}$. **Backprop**: define $\delta^{(\ell)} = \partial L/\partial z^{(\ell)}$: $\partial_{W^{(\ell)}} L = \delta^{(\ell)}(h^{(\ell-1)})^\top$, $\partial_{b^{(\ell)}} L = \delta^{(\ell)}$, $\delta^{(\ell-1)} = ((W^{(\ell)})^\top \delta^{(\ell)}) \odot \phi'(z^{(\ell-1)})$.
Activation: sigmoid/tanh saturate; ReLU $\max(0,z)$ keeps positive-side gradient; ELU/smooth variants improve negative side. **Subgradient**: ReLU at 0 uses any value in $[0,1]$ by convention.

1.3 CNN

Motivation: locality + weight sharing. Convolution:

$$y_{i,j,o} = \sum_{u,v,c} K_{u,v,c,o}\, x_{i+u,j+v,c} + b_o.$$

For input $H\times W\times C_{\text{in}}$, kernel $K_h\times K_w$, padding P, stride S: $H' = \lfloor (H+2P-K_h)/S\rfloor+1$, $W' = \lfloor (W+2P-K_w)/S\rfloor+1$. Params $K_hK_wC_{\text{in}}C_{\text{out}}+C_{\text{out}}$. **Scanning MLP**: same detector slides across space. **Pooling**: max/avg reduces spatial size and jitter. **Padding**: keeps boundary info; same padding for odd kernel uses $P=(K-1)/2$.

1.4 Optimization

GD: $x_{k+1} = x_k - \eta_k \nabla f(x_k)$. **$L$-smooth**: $\|\nabla f(x)-\nabla f(y)\|\le L\|x-y\|$; if C^2, $\nabla^2 f(x)\preceq LI$. It implies $f(y)\le f(x)+\nabla f(x)^\top(y-x)+\frac{L}{2}\|y-x\|^2$, so $0<\eta<2/L$ decreases the upper bound. **Strong convexity**: $f(y)\ge f(x)+\nabla f(x)^\top(y-x)+\frac{\mu}{2}\|y-x\|^2$; with L-smooth and C^2, $\mu I\preceq\nabla^2 f(x)\preceq LI$. GD has linear convergence $k=O(\kappa\log(1/\epsilon))$, $\kappa=L/\mu$.
Newton: from quadratic Taylor, $x_{k+1}=x_k-\eta[\nabla^2 f(x_k)]^{-1}\nabla f(x_k)$.
g_k = gradient on x_k. **Momentum**: $\Delta X^k = \beta\Delta X^{k-1}-\eta\nabla f(X^k)$, $X^{k+1}=X^k+\Delta X^k$.
Nesterov: compute gradient at lookahead $x_k+\beta(x_k-x_{k-1})$.
SGD / mini-batch: use one or a small batch of samples to estimate the full gradient; mini-batch reduces variance while keeping updates cheap. **Adaptive LR**: per-coordinate step sizes help convergence. **AdaGrad**: $s_k=s_{k-1}+g_k^2$, $x_{k+1}=x_k-\eta g_k/(\sqrt{s_k}+\epsilon)$; rarely updated dimensions have smaller $s_{k,i}$ and larger effective learning rate, but learning can decay too fast.
RMSProp: $s_k=\rho s_{k-1}+(1-\rho)g_k^2$.
Adam: $m_k=\beta_1 m_{k-1}+(1-\beta_1)g_k$, $v_k=\beta_2 v_{k-1}+(1-\beta_2)g_k^2$, $x_{k+1}=x_k-\eta\hat m_k/(\sqrt{\hat v_k}+\epsilon)$, $\hat m_k = m_k/(1-\beta_1^k)$.

1.5 Regularization and Architectures

Weight decay: L2 regularization stabilizes weights, $L_{\text{reg}}(w)=L_{\text{loss}}(w)+\alpha\|w\|_2^2$. **Early stopping**: stop when training improves but held-out validation worsens. **Dropout**: randomly zero units, prevents overfitting to particular neurons/weights, approximates ensemble and reduces co-adaptation; can be viewed as implicit L2. Two conventions: scale activations by keep-prob during training and leave inference unchanged, or leave training unchanged and multiply by keep-prob at inference. **Gradient clipping**: $g\leftarrow g\min(1,\tau/\|g\|)$.
Normalization: **GroupNorm**: batch-independent. **InstanceNorm**: batch-independent, suitable for generation tasks. **BatchNorm**: $\mu_B=\frac{1}{B}\sum_i z_i$, $\sigma_B^2=\frac{1}{B}\sum_i(z_i-\mu_B)^2$, $\hat z_i=\gamma\frac{z_i-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}+\beta$. Inference uses running averages because there is no mini-batch training distribution: $\mu_{\text{run}}\approx\frac{1}{N_{\text{batch}}}\sum_{\text{batch}}\mu_B$, $\sigma^2_{\text{run}}\approx\frac{1}{N_{\text{batch}}}\sum_{\text{batch}}\frac{B}{B-1}\sigma_B^2$. Backprop through a layer uses Jacobian transpose: $\nabla_X L = J^\top\nabla_Y L$. **PCA color jitter**: $[I_R,I_G,I_B]\leftarrow[I_R,I_G,I_B]+\alpha[P_R,P_G,P_B][\lambda_R,\lambda_G,\lambda_B]^\top$, $\alpha\sim\mathcal{N}(0,0.1)$; perturb RGB along principal color directions.
LayerNorm: normalize hidden dimension inside one sample; better for sequence models. Batch-independent, Particularly suitable for RNN. $h^t = f\left(\frac{g}{\sigma^t}\odot(a^t-\mu^t)+b\right)$, $\mu^t=\frac{1}{H}\sum_i a_i^t$, $\sigma^t=\sqrt{\frac{1}{H}\sum_i(a_i^t-\mu^t)^2}$.
ResNet: $h_{\ell+1}=h_\ell+F(h_\ell)$, residual path eases optimization. **DenseNet**: concatenate previous features. **FCN**: replace fully connected layer by convolution for dense prediction.

2 L4 Energy-Based Models

2.1 Energy View

Generative modeling: learn $P(X)$ or $P(X,y)=P(y)P(X|y)$ instead of $P(y|X)$. Generative models can be used to both classify and generate images
EBM: unnormalized score via energy

$$p_\theta(x) = \frac{e^{-E_\theta(x)}}{Z_\theta}, \quad Z_\theta = \int e^{-E_\theta(x)}dx.$$

Low energy means high probability; training and inference both need sampling or partition-function gradients. $\log p_\theta(x) = -E_\theta(x)-\log Z_\theta$, $\nabla_\theta \log p_\theta(x) = -\nabla_\theta E_\theta(x) - \frac{1}{Z_\theta}\nabla_\theta\left(\sum_{x'} e^{-E_\theta(x')}\right) = -\nabla_\theta E_\theta(x) + \mathbb{E}_{x'\sim p_\theta}[E_\theta(x')]$.
We can train an energy-based model without knowing the explicit density function (or normalizing factor Z). Such as score-matching, etc.

2.2 Hopfield Network

State $y_i\in\{-1,1\}$, local field $h_i=\sum_{j\neq i} w_{ij}y_j+b_i$. Update flips if $y_ih_i<0$.

$$E(y) = -\sum_{i<j} w_{ij}y_iy_j - \sum_i b_iy_i = -\frac{1}{2}y^\top Wy - b^\top y.$$

Asynchronous flip changes energy by $\Delta E = 2y_ih_i$; if $y_ih_i<0$, energy decreases. Finite states imply convergence to a local minimum. **Hebbian storage**: $w_{ij}=\frac{1}{Np}\sum_p y_i^p y_j^p$, $w_{ii}=0$. For N neurons, $K\le N/(4\log N)$ random patterns can be stable. Multiple patterns cause **spurious minima**; capacity is limited.
Objective for W: $W = \arg\min_W\left(\sum_{y\in P}E(y)-\sum_{y'\notin P}E(y')\right)$, $E(y)=-\frac{1}{2}y^\top Wy$. **Optimization view**: desired patterns should be valleys; short-run evolution can raise undesired valleys by contrastive updates.
SGD update rule for W: $W\leftarrow W+\eta\left(\mathbb{E}_{y\in P}[yy^\top]-\mathbb{E}_{y'}[y'y'^\top]\right)$; in practice sample desired y, initialize y' from a desired pattern, run 2–4 timesteps, update with $yy^\top - y'y'^\top$.

2.3 Boltzmann Machine

Stochastic Hopfield: energy $E(y)$ as Hopfield, $p(y)=\frac{1}{Z}e^{-E(y)}\propto e^{-E(y)}$.
Conditional Gibbs update for $\{-1,1\}$ units: for $h_i=\sum_j w_{ij}y_j+b_i$,

$$P(y_i=1|y_{-i}) = \frac{1}{1+e^{-2h_i}} = \sigma(2h_i), \quad h_i=\sum_j w_{ij}y_j+b_i.$$

$$\log\frac{P(y_i=1|y_{-i})}{1-P(y_i=1|y_{-i})} = 2h_i.$$

Stochastic evolution: initialize $y^{(0)}$, sample coordinates $y_i(t+1)\sim\text{Bernoulli}(\sigma(z_i(t)))$, repeat; recover pattern by last-M majority $\hat y_i = \mathbf{1}[\frac{1}{M}\sum_{t=L-M+1}^{L} y_i(t) > 0]$. Temperature T: $y_i(T)\sim\text{Bernoulli}(\sigma(z_i(T)/T))$, $T\leftarrow\alpha T$; annealing lowers T from exploration to exploitation.
BM Training: maximize $L(W)=\sum_{y\in P}\log P_\theta(y)$, θ means parameters.

$$P(y) = \frac{\exp(\frac{1}{2}y^\top Wy)}{\sum_{y'}\exp(\frac{1}{2}y'^\top Wy)}, \quad L(W) = \frac{1}{Np}\sum_{y\in P}\frac{1}{2}y^\top Wy - \log\sum_{y'} e^{\frac{1}{2}y'^\top Wy}.$$

$\nabla_{w_{ij}}\log p_\theta(y) = y_iy_j - \mathbb{E}_{p_\theta}[y_iy_j]$, $\nabla_{w_{ij}}L = \frac{1}{Np}\sum_{y\in P}y_iy_j - \frac{1}{M}\sum_{y'\in S}y'_iy'_j$. First term: compute average over P. Second term: sample from p_θ, compute average. **Sample method**: random initialize $y(0)$, cycle over i (coordinates) and sample from $P(y_i(t)|y_{-i}(t))$, after convergence select the final M states as samples. **Updating**: $w_{ij}\leftarrow w_{ij}+\eta\nabla_{w_{ij}}L(W)$. Learning = **positive phase** data correlation minus **negative phase** model correlation.
With hidden neurons: $y=(v,h)$

$$\nabla_{w_{ij}}L = \frac{1}{|P|}\sum_{v\in P}\mathbb{E}_{h|v}[y_iy_j] - \mathbb{E}_{y'\sim p_W}[y'_iy'_j].$$

Use conditioned samples S_c by clamping visible v and Gibbs-sampling hidden h, and free samples S by running all variables: $\nabla_{w_{ij}}L = \frac{1}{NpK}\sum_{y\in S_c}y_iy_j - \frac{1}{M}\sum_{y'\in S}y'_iy'_j$.
Boltzmann for classification: $y=(v,h,c)$, c: visible, one-hot vector

2.4 RBM and Contrastive Divergence

RBM: bipartite v,h; no visible-visible or hidden-hidden edges: $E(v,h)=-v^\top Wh-b^\top v-c^\top h$. Conditionals factorize: $P(h_j=1|v)=\sigma(c_j+\sum_i w_{ij}v_i)$, $P(v_i=1|h)=\sigma(b_i+\sum_j w_{ij}h_j)$, $\sigma(z)=\frac{1}{1+e^{-z}}$. Sampling: $v^{(0)}\to h^{(0)}\to v^{(1)}\to h^{(1)}\to\cdots$

Maximum likelihood estimate: $\nabla_{w_{ij}}L = \frac{1}{NpK}\sum_{v\in P}v_i^0h_j^0 - \frac{1}{M}\sum v_i^\infty h_j^\infty$.
CD-1 approximation:

$$\nabla_{w_{ij}}L \approx \frac{1}{Np}\sum_{v\in P}(v_i^0h_j^0 - v_i^1h_j^1).$$

CD-k: start at data $v^{(0)}$, run k Gibbs steps, update $W\propto v^{(0)}h^{(0)\top} - v^{(k)}h^{(k)\top}$.

2.5 Sampling

Inverse CDF: sample $u\sim U[0,1]$, choose category by cumulative mass. **Box-Muller**: Gaussian from uniform:

$$z_1 = \sqrt{-2\log u_1}\cos(2\pi u_2), \quad z_2 = \sqrt{-2\log u_1}\sin(2\pi u_2).$$

Importance sampling: $\mathbb{E}_p[f(x)] = \mathbb{E}_q[f(x)p(x)/q(x)]$. Good q must cover p's high-mass region. **MCMC**: **Detailed balance** $\pi(x)T(x\to x') = \pi(x')T(x'\to x)$ implies stationarity of π. **Ergodic**: can visit any desired state with positive probability and has a unique stationary distribution; a strong minorization form is $\min_{z,z':\pi(z')>0}T(z\to z')/\pi(z')=\delta>0$. **Metropolis Hastings Algo.**: Construct Markov chain with stationary π (desired distribution). Current x, draw $x'\sim q(x'|x)$ (called proposal), transition to x' with prob. $\alpha=\min\left(1,\frac{\pi(x')q(x|x')}{\pi(x)q(x'|x)}\right)$, otherwise stays at x. Choice of q: random proposal $q(x'|x)$ =noise (e.g. Gaussian, then it relies on $|x-x'|$), then q terms in α vanishes.
For EBM, partition Z cancels.
Gibbs: sample a single coordinate per step from $q(s_i\to s_i')=p(s_i'|s_{j\neq i})$; it is MH with acceptance probability 1.
SGLD: $x_{t+1}=x_t-\frac{\eta}{2}\nabla_x E_\theta(x_t)+\sqrt{\eta}\xi_t$.

3 L5 Generative Adversarial Networks

3.1 Implicit Generative Model

Instead of density $p_\theta(x)$, learn sampler $x=G_\theta(z)$, $z\sim p_z$. Likelihood-free learning. Need differentiable distance between generated and real distributions. GAN learns this distance with a discriminator.
GAN: generator $G_\theta(z)$, $z\sim p(z)=N(0,I)$; discriminator $D_\phi(x)$, classify the data is from p_{data} or G_θ. Objective: $L=\min_\theta\max_\phi V(D_\phi,G_\theta)$,

$$V(D_\phi,G_\theta) = \mathbb{E}_{x\sim p_{\text{data}}}\log D_\phi(x) + \mathbb{E}_{z\sim p_z}\log(1-D_\phi(G_\theta(z))).$$

Dataset: $\{(x,1):x\sim p_{data}\}\cup\{(\hat x,0):\hat x\sim G_\theta(z)\}$.
Train D: $L(\phi)=\mathbb{E}_{x\sim p_{\text{data}}}\log D_\phi(x)+\mathbb{E}_{z\sim p_z}\log(1-D_\phi(G(z)))$.
Train G: $L(\theta)=\mathbb{E}_{z\sim p(z)}[\log D_\phi(G_\theta(z))]$.
For fixed G, the optimal D is $D^\star(x)=\frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_G(x)}$. Substitute: $V(D^\star,G)=-\log 4+2\,\text{JSD}(p_{\text{data}}\|p_g)$. **JSD**: $\text{JSD}(p\|q)=\text{KL}(p\|\frac{p+q}{2})+\text{KL}(q\|\frac{p+q}{2})$, ≥ 0, $\sqrt{\text{JSD}}$ satisfies triangular inequality.
$2\text{KL}(p\|q)\ge(\int|p(x)-q(x)|dx)^2$.
Non-saturating generator loss $-\mathbb{E}_z\log D(G(z))$ gives stronger gradients.

3.2 Evaluation and Failure Modes

Mode collapse: generator covers few modes. For multimodal p, reverse KL $\text{KL}(q\|p)=\mathbb{E}_q\log(q/p)$ and JSD often prefer one high-quality mode; forward KL $\text{KL}(p\|q)=\mathbb{E}_p\log(p/q)$ penalizes missing modes and is mode-covering. JSD: $\frac{1}{2}\text{KL}(p\|m)+\frac{1}{2}\text{KL}(q\|m)$, $m=(p+q)/2$. **Training instability**: moving target game can oscillate; discriminator too strong/simple $\Rightarrow$ generator gradient vanishes.
Inception Score:

$$\text{IS} = \exp\left(\mathbb{E}_{x\sim G}\text{KL}(f(y|x)\|p_f(y))\right), \quad p_f(y)=\mathbb{E}_{x\sim G}f(y|x).$$

Since $\mathbb{E}_x\text{KL}(f(\cdot|x)\|p_f) = -\mathbb{E}_x H(f(\cdot|x))+H(p_f)$, high IS needs confident images and diverse classes. IS never uses p_{data}, so it cannot detect memorization. **FID**: Compute μ_p,Σ_p and μ_G,Σ_G for $p_{data},G(z)$ using Inception-v3 pool3 layer (2048-d); compare Gaussian feature statistics: $\|\mu_p-\mu_G\|_2^2+\text{tr}(\Sigma_p+\Sigma_G-2(\Sigma_p\Sigma_G)^{1/2})$.. Lower FID score means better generators.

3.3 GAN Training Techniques

Feature Matching: train G to match discriminator feature statistics:

$$L_G(\theta) = \left\|\mathbb{E}_{\hat x\sim G}f(\hat x;\phi)-\mathbb{E}_{x\sim p_{\text{data}}}f(x;\phi)\right\|_2^2,$$

where $f(x;\phi)$ is the final activation layer of D. **Other tricks**: One-Sided Label Smoothing; Historical Averaging; Minibatch Discrimination; Virtual Batch Normalization.

3.4 WGAN Family

JSD can be flat when supports do not overlap. Wasserstein-1: $W(p,q)=\sup_{\|f\|_L\le 1}\mathbb{E}_{x\sim p}f(x)-\mathbb{E}_{x\sim q}f(x)$.
WGAN: critic f_ϕ, no D. WGAN objective:

$$\min_G\max_{D:|D|_L\le 1}\mathbb{E}_{x\sim p_{\text{data}}}[D(x)]-\mathbb{E}_{\hat x\sim p_G}[D(\hat x)].$$

Equivalently maximize critic $L(\phi)=\mathbb{E}_{x\sim p_{\text{data}}}f_\phi(x)-\mathbb{E}_{\hat x\sim p_G}f_\phi(\hat x)$ and train G to reduce the critic gap.
Lipschitz constraint: weight clipping, spectral norm, or **WGAN-GP**: $\bar x\leftarrow(1-\epsilon)x+\epsilon\hat x$, $\epsilon\sim\text{Unif}(0,1)$, $L(\phi)=\mathbb{E}_{x\sim p_{data}}[f_\phi(x)]-\mathbb{E}_{\hat x\sim p_G}[f_\phi(\hat x)]+\lambda\mathbb{E}_{\bar x}(\|\nabla_{\bar x}f_\phi(\bar x)\|_2-1)^2$.
DCGAN: conv/deconv, batchnorm, stable architecture. **BigGAN**: scale + class conditioning + truncation trick. **GigaGAN**: text-to-image GAN scaling. **R3GAN**: regularized robust objective.

3.5 Variants and Applications

Conditional GAN: $G(z,c)$, $D(x,c)$. **Semi-supervised GAN**: discriminator has $K+1$ classes, K real labels plus fake. **BiGAN/ALI**: learn encoder $E(x)$ with discriminator over (x,z) pairs. Idea: $(z,G(z))$ and $(E(x),x)$ from same distribution. gives representation learning. **Pix2Pix**: paired image-to-image translation with conditional GAN + reconstruction loss. **CycleGAN**: unpaired translation using cycle consistency $G:X\to Y$, $F:Y\to X$, $\|F(G(x))-x\|+\|G(F(y))-y\|$. **DragGAN**: interactive image manipulation by optimizing latent/code with handle-point constraints.

4 L6 Flow and VAE

4.1 Normalizing Flow

Invertible differentiable generator $x=G_\theta(z)$, transform a simple distribution to the data distribution. $\log p_X(x)=\log p_Z(G^{-1}(x))+\log\left|\det\frac{\partial G^{-1}}{\partial x}\right|$.
Composition: $z_K=G(z_0)=f_K\circ\cdots\circ f_1(z_0)$, $\log p_X(z_K)=\log p_Z(z_0)-\sum_k\log|\det\frac{\partial f_k}{\partial z_k}|$. Flow design needs invertible layer, efficient inverse, efficient compute determinant of Jacobi.
Coupling: split $x=(x_a,x_b)$, to (y_a,y_b), $y_a=x_a$, $y_b=x_b\odot e^{s(x_a)}+t(x_a)$, $\log|\det J|=\sum s(x_a)$. s,t: learnable. sometimes needs premutaion among coordinates or linear transformation.
Glow: invertible 1×1 convolution mixes channels. Since the same channel-mixing matrix W is applied at each spatial location,

$$\log\left|\det\frac{d\,\text{conv2D}(h;W)}{dh}\right| = HW_{\text{spatial}}\log|\det W|.$$

LU parameterization $W=PL(U+\text{diag}(s))$ makes determinant cheap: $\log|\det W|=\sum_i\log|s_i|$. **Dequantization**: add uniform noise to discrete pixels so continuous density is well-defined. **Continuous NF / Neural ODE**: $\frac{dz(t)}{dt}=f_\theta(z(t),t)$, $\log p(z(t_1))=\log p(z(t_0))-\int_{t_0}^{t_1}\text{tr}\left(\frac{\partial f_\theta}{\partial z(t)}\right)dt$.
Prove a Normalizing Flow: 1. invertible; 2. easily-tractable Jacobi matrix,

4.2 GMM and EM

GMM responsibility: for K mixture components,

$$\gamma_{ij} := p(z_i=j|x_i) = \frac{\pi_j\mathcal{N}(x_i|\mu_j,\Sigma_j)}{\sum_{\ell=1}^K\pi_\ell\mathcal{N}(x_i|\mu_\ell,\Sigma_\ell)}.$$

GMM EM lower bound:

$$\sum_i\log\sum_j\gamma_{ij}\frac{\pi_j\mathcal{N}(x_i|\mu_j,\Sigma_j)}{\gamma_{ij}} \ge \sum_i\sum_j\gamma_{ij}\log\frac{\pi_j\mathcal{N}(x_i|\mu_j,\Sigma_j)}{\gamma_{ij}}.$$

E-step fixes parameters and maximizes over γ:

$$\gamma^{(t+1)} = \arg\max_{\gamma_{ij}\ge 0,\sum_j\gamma_{ij}=1}\sum_{i,j}\gamma_{ij}\log\pi_j^{(t)}\mathcal{N}(x_i|\mu_j^{(t)},\Sigma_j^{(t)})-\sum_{i,j}\gamma_{ij}\log\gamma_{ij}.$$

M-step fixes γ and maximizes

$$(\pi,\mu,\Sigma)^{(t+1)} = \arg\max_{\pi,\mu,\Sigma}\sum_{i,j}\gamma_{ij}^{(t+1)}\log\pi_j\mathcal{N}(x_i|\mu_j,\Sigma_j).$$

4.3 Evidence Lower Bound (ELBO)

Latent-variable likelihood $\log p_\theta(x)=\log\sum_z p_\theta(x,z)$. Jensen:

$$\log p_\theta(x) = \log\mathbb{E}_q\frac{p_\theta(x,z)}{q(z)} \ge \mathbb{E}_q\log p_\theta(x,z)+H(q) = \mathcal{L}(q,\theta).$$

Gap: $\log p_\theta(x) - \mathcal{L}(q,\theta) = \mathrm{KL}(q(z)\|p_\theta(z|x))$. Equivalently

$$\log p_\theta(x) \geq \int q_\phi(z|x) \log p_\theta(x|z)\, dz - \mathrm{KL}(q_\phi(z|x)\|p(z)),$$

with gap $\mathrm{KL}(q_\phi(z|x)\|p_\theta(z|x))$. EM is coordinate ascent on this lower bound: E-step sets $q(z) = p_{\theta^{old}}(z|x)$, M-step maximizes $\mathbb{E}_q \log p_\theta(x,z)$.

4.4 VAE

Encoder $q_\chi(z) = \mathcal{N}(\mu_\theta(x), \sigma_\theta(x))$, decoder $p_\psi(x|z) = \mathcal{N}(\psi(z), \beta I)$, prior $p(z)$.
ELBO: $\log p(x) \geq \int q_\chi(z) \log p_\psi(x|z)\, dz - \mathrm{KL}(q_\chi(z)\|p(z))$. With Gaussian decoder,

$$\mathrm{ELBO} = -\frac{1}{2\beta}\mathbb{E}_{q_\chi(z)}\|\psi(z) - x\|_2^2 - \mathrm{KL}(q_\chi(z)\|p(z)) + C.$$

Reparameterization: $z = \mu_\theta(x) + \sigma_\theta(x)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$. **Gaussian KL**: for diagonal Gaussians $p = \mathcal{N}(\mu_1, \sigma_1^2)$, $q = \mathcal{N}(\mu_2, \sigma_2^2)$,

$$\mathrm{KL}(p\|q) = \log\frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}.$$

For $p(z) = \mathcal{N}(0, I)$, $\mathrm{KL}(q_\chi(z)\|p(z)) = \frac{1}{2}\|\mu_\theta(x)\|^2 + \frac{1}{2}\sigma_\theta^2(x) - \log\sigma_\theta(x) + C$.
Terms: reconstruction = data fidelity; KL = latent regularization. In β-**VAE**, large β enforces latent variables to be **less** correlated with each other:

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \beta\,\mathrm{KL}(q_\phi(z \mid x)\|p(z)).$$

PixelVAE/NVAE: hierarchical latent variables improve global coherence and likelihood.

5 L7 Diffusion and Flow Matching

5.1 DDPM

Forward: gradually add Gaussian noise,

$$q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1-\beta_t}\,x_{t-1}, \beta_t I), \quad \alpha_t = 1 - \beta_t, \quad \bar\alpha_t = \prod_{s=1}^{t}\alpha_s.$$

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar\alpha_t}x_0, (1-\bar\alpha_t)I), \quad x_t = \sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\epsilon.$$

Reverse: learn denoising transitions,

$$p_\theta(x_{0:T}) = p(x_T)\prod_{t=1}^{T} p_\theta(x_{t-1}|x_t), \quad p(x_T) = \mathcal{N}(0, I),$$

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t,t), \sigma_t^2 I).$$

Loss / ELBO identity: reuse the Gaussian-product identity

$$\log q(x_t|x_{t-1}) + \log q(x_{t-1}|x_0) = \log q(x_{t-1}|x_t, x_0) + \log q(x_t|x_0).$$

The generic VAE-style decomposition is

$$-\log p(x) = -\int q_\phi(z)\log p(x|z)\, dz + \mathrm{KL}(q_\phi(z)\|p(z)) - \mathrm{KL}(q_\phi(z)\|p(z|x)).$$

True posterior:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde\mu_t(x_t, x_0), \tilde\beta_t I), \quad \tilde\beta_t = \frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}\beta_t,$$

$$\tilde\mu_t = \frac{\sqrt{\bar\alpha_{t-1}}\beta_t}{1-\bar\alpha_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}x_t.$$

Gaussian KL reduces each timestep to mean matching:

$$L_{t-1} = \mathbb{E}_q\left[\frac{1}{2\sigma_t^2}\|\tilde\mu_t(x_t,x_0) - \mu_\theta(x_t,t)\|^2\right] + C.$$

Noise prediction: since

$$\tilde\mu_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\epsilon\right), \quad \mu_\theta = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\epsilon_\theta(x_t,t)\right),$$

$$L_{t-1} = \mathbb{E}_{x_0,\epsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar\alpha_t)}\|\epsilon - \epsilon_\theta(x_t,t)\|^2\right] + C.$$

Training input is (x_t, t) and target is noise ϵ; $\epsilon_\theta(x_t,t)$ is usually a U-Net with time encoding. **Sampling** from $x_T \sim \mathcal{N}(0, I)$:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\epsilon_\theta(x_t,t)\right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I).$$

5.2 Score-Based Models

Score: learn score $\nabla_x \log p(x)$. **Langevin dynamics**: use only score. $x_{t+1} = x_t + \eta\nabla_x \log p(x_t) + \sqrt{2\eta}\epsilon_t, \epsilon_t \sim \mathcal{N}(0, I)$. continuous form: $\mathrm{d}x_t = \nabla_x \log p(x_t)\mathrm{d}t + \sqrt{2}\mathrm{d}w_t, \mathrm{d}w \sim \mathcal{N}(0, d_t)$.
NCSN: noise conditional score network. $x_i = x_0 + \sigma_i\epsilon$, learn $s_\theta(x_i, \sigma_i)$ to estimate $\nabla_x \log p(x_i)$.
$x_t = \alpha_t x_0 + \sigma_t\epsilon$, score $s_\theta(x_t,t)$? MSE loss $L = \mathbb{E}_{x_t}\|s_\theta(x_t,t) - \nabla_{x_t}\log p(x_t)\|^2$. $\mathbb{E}_{x_t}[s_\theta(x_t,t)^\top \nabla_{x_t}\log p(x_t)] = \mathbb{E}_{x_t}[\nabla\cdot s_\theta(x_t,t)]$, using $\nabla\cdot(sp) = (\nabla\cdot s)p + s^\top\cdot\nabla p$
Denoising score matching: $\nabla_{x_t}\log p(x_t) = \mathbb{E}_{x_0\sim p(\cdot|x_t)}[\nabla_{x_t}\log p(x_t|x_0)]$
$\mathbb{E}_{x_t}\|s_\theta(x_t,t) - \nabla_{x_t}\log p(x_t|x_0)\|^2$,

$$L_{DSM} = \mathbb{E}_{x_0,t,x_t}\|s_\theta(x_t,t) - \nabla_{x_t}\log q(x_t|x_0)\|^2,$$

where $x_t \sim q(x_t|x_0)$ is tractable. If $x_t = a_t x_0 + \sigma_t\epsilon$, then $\nabla_{x_t}\log q(x_t|x_0) = -\frac{x_t - a_t x_0}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}$. Let $s_\theta(x_t,t) = -\frac{\epsilon_\theta(x_t,t)}{\sigma_t}$; loss matches DDPM.
SDE: $dx = f(x,t)dt + g(t)dW_t$,
reverse: $dx = [f(x,t) - g(t)^2\nabla_x\log p_t(x)]dt + g(t)d\bar W_t$.
A equivalent ODE (same distribution when same t): Probability-flow ODE: $dx = \left[f(x,t) - \frac{1}{2}g(t)^2\nabla_x\log p_t(x)\right]dt$. Likelihood along ODE: $\log p_0(x_0) = \log p_T(x_T) + \int_0^T \mathrm{tr}\left(\frac{\partial f_\theta}{\partial x_t}\right)dt$.
Inference: use learned score to solve SDE/ODE using a solver. Hence #sample-steps in training and inference can be different.
DDIM: sample according to schedule

5.3 DPM-Solver and Applications

DPM solver 1 order DPM = DDIM.
DPM-Solver integrates diffusion ODE with high-order exponential integrators, reducing sampling steps. **Latent diffusion**: diffuse in autoencoder latent space, cheaper than pixel space. **DiT/SORA**: transformer over spacetime latent patches. **DALL-E 2**: CLIP prior + diffusion decoder. **DALL-E 3**: synthetic captions improve prompt following. **SDEdit**: add noise to input image then denoise under prompt. **ControlNet**: extra condition branch injects edges/depth/pose. **Logit-normal sampling**: biases timestep sampling toward informative middle region.

5.4 Flow Matching

Continuous flow $dx_t/dt = v_\theta(x_t,t)$. Objective: $\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{t,x_t}\|v_\theta(x_t,t) - u_t(x_t)\|^2$. Conditional path from noise x_0 to data x_1: $x_t = (1-t)x_0 + tx_1$, target $u_t = x_1 - x_0$. Conditional Flow Matching (CFM) samples pairs and regresses conditional velocity; marginal vector field is recovered in expectation. **Non-uniqueness**: many vector fields induce same marginals; choice affects training variance and sampling trajectory. **SD3**: flow target often parameterizes velocity between noise/data under continuous timestep.

6 L8–9 Sequence Modeling

6.1 Sequence Data and Tokenization

Sequence data includes language, audio, action; images/DNA/protein can be serialized. Autoregressive factorization: $p(x_{1:T}) = \prod_{t=1}^{T} p(x_t|x_{<t})$.
Tokenization: text → basic units → token IDs. Byte-Pair Encoding (BPE) repeatedly merges frequent adjacent symbols; balances vocabulary size and sequence length. **Embedding**: token ID $i \to e_i = W_E[i]$; positional info is needed because token embeddings alone are order-free.

6.2 Autoregressive CNN and Pixel Models

WaveNet: causal/dilated temporal convolution expands receptive field without recurrence. **PixelCNN**: image likelihood $p(x) = \prod_i p(x_i|x_{<i})$. under raster order; masked convolution prevents access to future pixels. Gated PixelCNN uses gates to improve expressiveness.

6.3 RNN and LSTM

RNN: $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad y_t = W_{hy}h_t + b_y$. BPTT multiplies many Jacobians; singular values < 1 vanish, > 1 explode.
LSTM: $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f), \quad i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$,
$\tilde c_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde c_t$,
$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o), \quad h_t = o_t \odot \tanh(c_t)$. Additive cell path gives controllable memory; forget gate decides retention.

6.4 Attention and Transformer

For token t: $q_t = W_Q x_t$, $k_j = W_K x_j$, $v_j = W_V x_j$, $\alpha_{tj} = \frac{\exp(q_t^\top k_j/\sqrt{d_k})}{\sum_i \exp(q_t^\top k_i/\sqrt{d_k})}$, $y_t = \sum_j \alpha_{tj} v_j$. Matrix form: $\mathrm{Attn}(Q,K,V) = \mathrm{softmax}(QK^\top/\sqrt{d_k})V$.
Multi-head: parallel attention subspaces, concatenate and project. **Causal mask**: block future tokens. **Transformer block**: attention + MLP + residual + normalization. Complexity $O(T^2 d)$; FlashAttention reduces memory IO, Mamba/SSM explores linear-time sequence mixing.

6.5 Objectives and Case Studies

Objectives: autoregressive LM, masked/denoising, contrastive predictive. **Recipe**: pretraining, fine-tuning, low-rank tuning. **GPT**: decoder-only causal Transformer; next-token pretraining gives in-context learning with scale. **ViT**: image patches as tokens. **Swin**: shifted local windows for hierarchical vision. **CLIP**: contrastive image-text alignment,

$$L = -\frac{1}{2}\left[\frac{1}{N}\sum_i \log\frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ij}/\tau}} + \frac{1}{N}\sum_i \log\frac{e^{s_{ii}/\tau}}{\sum_j e^{s_{ji}/\tau}}\right].$$

SigLIP: pairwise sigmoid loss avoids global softmax. **Whisper**: audio Transformer trained on large-scale speech. **MAR**: masked autoregressive modeling for images.

7 L10 Modern Large Models

7.1 Pretraining, Scaling, Prompting

Pipeline: Pretraining → Base GPT → SFT → RLHF → ChatGPT-like. Pretraining builds capability by next-token prediction; post-training reshapes behavior into instruction-following. Base GPT tends to continue text; chat model tries to answer the user. **Scaling laws**: held-out loss decreases smoothly as model size N, data D, and compute C grow, approximately by power law. The important lesson is bottleneck balance: scale parameters, tokens, and compute together. Useful heuristics: data should grow with model size ($D \propto N^{0.74}$); compute-optimal training spends extra compute mostly on larger N, somewhat on batch B, and barely on serial steps S. **Prompting**: zero-shot/few-shot puts task description/examples in context without parameter updates. GPT-3 made in-context learning practical; instruction tuning makes this behavior reliable by training on natural-language tasks.

7.2 SFT and RLHF

SFT: learn from human demonstrations (x, y) by next-token imitation:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\mathbb{E}_{\mathcal{D}_{\mathrm{SFT}}}\sum_t \log\pi_\theta(y_t \mid x, y_{<t}).$$

It teaches desired response style but only uses positive examples, so it cannot directly rank two acceptable answers. **Reward model**: pairwise preference $y_w \succ y_l$ teaches scalar reward $r_\phi(x,y)$:

$$P_\phi(y_w \succ y_l \mid x) = \sigma(r_\phi(x,y_w) - r_\phi(x,y_l)).$$

$$\mathcal{L}_{\mathrm{RM}} = -\mathbb{E}\log\sigma(r_\phi(x,y_w) - r_\phi(x,y_l)).$$

RLHF: optimize policy reward while staying near reference/SFT model:

$$\max_\theta \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x,y) - \beta\mathrm{KL}(\pi_\theta(\cdot \mid x)\|\pi_{\mathrm{ref}}(\cdot \mid x))\right].$$

Why KL: prevents reward hacking and language-quality drift. **Pretrain vs post-train**: pretraining compresses web text distribution; post-training optimizes human utility/preference over responses.

7.3 Hallucination, RAG, Reasoning

Hallucination: SFT can teach a confident answer even when the base model lacks knowledge. Preference/RLHF can reward uncertainty awareness, e.g. prefer "I don't know" over wrong entities. Negative samples (x, y^+, y^-) are valuable because they mark bad-but-plausible behavior. **RAG**: combine parametric memory in θ with retrieved non-parametric memory z:

$$p(y \mid x) \approx \sum_{z\in\mathrm{TopK}(x)} p_\eta(z \mid x)\, p_\theta(y \mid x, z).$$

RAG helps latest knowledge without retraining, but retrieval is not the same as reasoning. **LRM**: reasoning model spends test-time compute on internal thinking before final answer:

$$x \to z_{1:m} \to y.$$

CoT prompting elicits intermediate reasoning through prompt examples; reasoning RL trains the behavior with verifier reward. Thinking labels are not unique, so RL can reward final correctness without prescribing one chain. Trajectory:

$$\tau = (z_1, \ldots, z_m, y_1, \ldots, y_T), \quad R(x,\tau) = \begin{cases} +1, & V(x,y) = \text{correct}, \\ -1, & V(x,y) = \text{incorrect}. \end{cases}$$

Agent loop: reasoning can extend to environment interaction with actions:

$$s_t = (x, \text{memory}, \text{observations}_{<t}), \quad a_t \in \{\text{search}, \text{browse}, \text{code}, \text{write}, \ldots\}.$$

7.4 Multimodal Models

Grounding: multimodal models condition on text plus image/video/audio/interface inputs:

$$p_\theta(y \mid x_{\text{text}}, x_{\text{image}}, x_{\text{video}}, \ldots).$$

Flamingo: extends GPT-style few-shot prompting to interleaved text-image/video input:

$$p_\theta(y \mid t_1, v_1, t_2, v_2, \ldots, t_k, v_k, t_{k+1}).$$

Architecture: frozen vision encoder + frozen LM; Perceiver Resampler compresses visual features; gated cross-attention injects vision while protecting frozen LM. **LLaVA**: visual instruction tuning samples $(I, x_{\text{instruction}}, y_{\text{assistant}})$; CLIP features are projected into LM embedding space:

$$F_I = f_{\text{CLIP}}(I), \quad Z_I = F_I W, \quad [Z_I; E(x_1); \ldots; E(x_T)].$$

Two-stage training: first align visual features by training projection W; then freeze vision encoder and tune projection + LM for assistant behavior. **Unified multimodal/BAGEL**: one decoder for understanding/generation/editing/reasoning on interleaved data $t_1, I_1, t_2, I_2, \ldots, V_1, t_k$; ViT pathway supports understanding, VAE pathway supports generation/reconstruction. Emergence order: understanding → generation → editing → reasoning.

8 Cross-Lecture Map

Density models: EBM defines $p \propto e^{-E}$ but sampling is hard; flow gives exact likelihood via invertible map; VAE gives ELBO; diffusion learns reverse denoising; flow matching learns vector field directly. **Implicit models**: GAN learns sampler without tractable likelihood. **Sequence models**: Transformer turns all modalities into token sequences. **Large models**: pretraining builds capability, post-training aligns behavior, inference-time compute and multimodal grounding extend usability.